

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

VICERRECTORADO ACADÉMICO

DIRECCIÓN DE DESARROLLO ACADÉMICO



FACULTAD: INFORMATICA Y ELECTRONICA

CARRERA: SOFTWARE

DISEÑO ARQUITECTÓNICO DE LA APLICACIÓN

CODIGO DE LA ASIGNATURA

SOF1TP49

APLICACIONES INFORMÁTICAS II

NOMBRES Y CÓDIGO

SHIRLEY EDIZA CHELA LLUMIGUANO

7322

1. Introducción

El presente documento describe el diseño arquitectónico de la aplicación móvil desarrollada para fortalecer la destreza *speaking* mediante el uso de Machine Learning.

Para ello, se implementó una **arquitectura en N capas**, que organiza el sistema en tres niveles: **Presentación, Negocio y Datos**, permitiendo separar responsabilidades y facilitar el mantenimiento, escalabilidad y reutilización del software.

La arquitectura integra Flutter como aplicación móvil, FastAPI para la lógica de negocio y Firebase como plataforma de autenticación y almacenamiento de datos.

2. Estilo arquitectónico

Para el desarrollo de la aplicación se seleccionó una **arquitectura en N capas**, debido a que permite organizar el sistema mediante la separación de responsabilidades entre la interfaz de usuario, la lógica de negocio y la gestión de datos. Esta estructura facilita el mantenimiento, la escalabilidad y la reutilización del software, además de permitir que cada capa funcione de manera independiente.

La **capa de presentación** fue desarrollada en Flutter y es la encargada de gestionar la interacción entre los usuarios y la aplicación. En esta capa se implementan las interfaces para el registro e inicio de sesión, el ingreso a cursos, la interacción con el asistente virtual y la visualización del progreso y los resultados obtenidos.

La **capa de negocio** fue implementada mediante FastAPI, donde se centraliza el procesamiento de las solicitudes provenientes de la aplicación móvil. En esta capa se ejecuta la lógica del sistema y se integra el modelo preentrenado de Machine Learning encargado del análisis de la pronunciación y la gramática, generando la retroalimentación automática para el estudiante.

La **capa de datos** utiliza Firebase como plataforma para la gestión de la información del sistema. Mediante Firebase Authentication se administra la autenticación de los usuarios; Cloud Firestore permite almacenar usuarios, cursos, conversaciones y resultados; mientras que Firebase Storage gestiona los archivos de audio generados durante las prácticas de speaking.

Esta arquitectura permite que cada una de las capas cumpla funciones específicas e independientes, favoreciendo un sistema modular, mantenible y preparado para futuras ampliaciones, además de garantizar una comunicación eficiente entre los diferentes componentes de la aplicación.

3. Patrones arquitectónicos utilizados

En el desarrollo de la aplicación se implementaron diferentes patrones de diseño que complementan la arquitectura en N capas, permitiendo organizar el código de forma modular, reducir el acoplamiento entre componentes y facilitar el mantenimiento y la evolución del sistema.

En primer lugar, se emplea el **patrón MVVM (Model–View–ViewModel)** en la aplicación desarrollada en Flutter. Este patrón organiza la capa de presentación separando la interfaz de usuario (View), la lógica de presentación (ViewModel) y los modelos de datos (Model). De esta manera, la interfaz permanece desacoplada de la lógica de la aplicación, facilitando el mantenimiento y la reutilización del código.

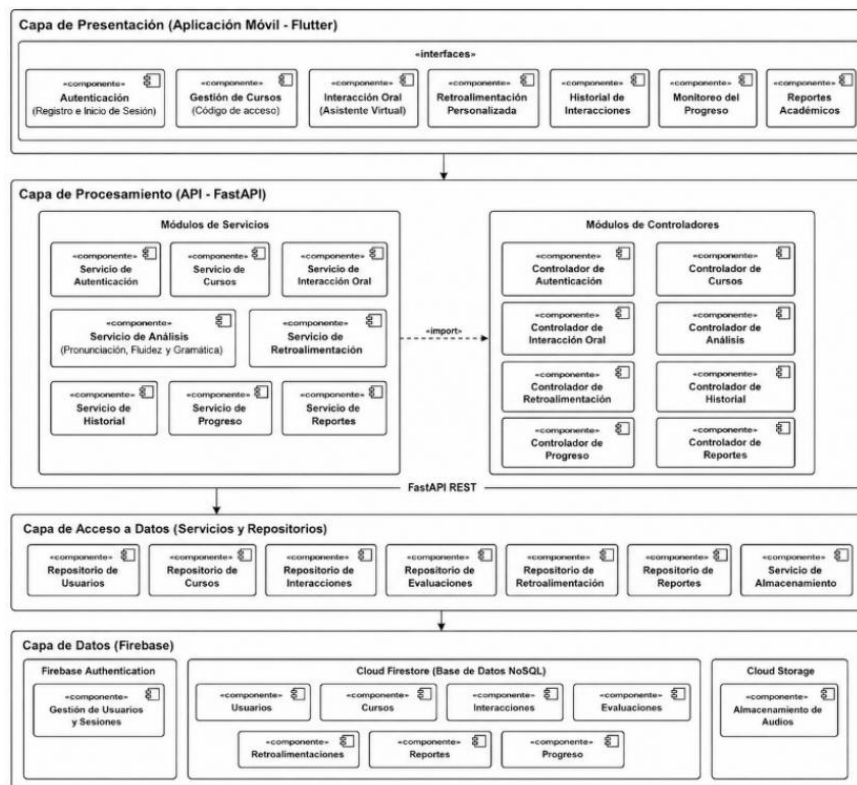
En segundo lugar, se implementa el **patrón Repository**, el cual actúa como intermediario entre la lógica de la aplicación y las diferentes fuentes de datos. Este patrón centraliza el acceso a la información proveniente de Firebase y de los servicios desarrollados en FastAPI, evitando dependencias directas entre la interfaz de usuario y la capa de datos.

Asimismo, la aplicación utiliza el **patrón Cliente–Servidor**, donde la aplicación móvil desarrollada en Flutter funciona como cliente y el servidor implementado en FastAPI procesa las solicitudes relacionadas con la autenticación, la gestión de información y la comunicación con el modelo preentrenado de Machine Learning. Esta organización permite distribuir las responsabilidades entre el cliente y el servidor, mejorando la organización y el rendimiento del sistema.

La integración de estos patrones de diseño permite construir una aplicación modular, desacoplada y preparada para futuras ampliaciones, manteniendo una adecuada organización del código y facilitando su mantenimiento.

4. Diagrama arquitectónico

A continuación, se presenta el diagrama arquitectónico del sistema, el cual ilustra la estructura general de la aplicación, los componentes principales y la interacción entre ellos.



5. Descripción del diagrama

El diagrama arquitectónico presenta la estructura general de la aplicación, organizada mediante una **arquitectura en N capas**, la cual separa las responsabilidades del sistema en los niveles de presentación, procesamiento, acceso a datos y persistencia. Esta organización facilita el mantenimiento, la escalabilidad y la comunicación entre los diferentes componentes de la aplicación.

- **Capa de Presentación (Aplicación móvil – Flutter):** corresponde a la interfaz con la que interactúan los usuarios del sistema. En esta capa se implementan los módulos de autenticación, gestión de cursos mediante códigos de acceso, interacción oral con el asistente virtual, retroalimentación personalizada, historial de interacciones, monitoreo del progreso y generación de reportes académicos. Su función es capturar las solicitudes del usuario y mostrar la información procesada por las capas inferiores.
- **Capa de Procesamiento (API – FastAPI):** concentra la lógica de negocio del sistema. Está conformada por módulos de servicios y controladores encargados de gestionar la autenticación, los cursos, la interacción oral, el análisis de pronunciación y gramática, la generación de retroalimentación, el historial, el progreso y los reportes. En esta capa también se integra el modelo preentrenado

de Machine Learning utilizado para analizar las respuestas del estudiante y generar la retroalimentación correspondiente.

- **Capa de Acceso a Datos (Servicios y Repositorios):** actúa como intermediaria entre la lógica de negocio y la base de datos. Está formada por repositorios especializados para la gestión de usuarios, cursos, interacciones, evaluaciones, retroalimentaciones, reportes y almacenamiento de archivos. Esta capa centraliza el acceso a la información, evitando dependencias directas entre la lógica de negocio y la persistencia de datos.
- **Capa de Datos (Firebase):** constituye el nivel de persistencia del sistema. Firebase Authentication administra el registro e inicio de sesión de los usuarios; Cloud Firestore almacena la información relacionada con usuarios, cursos, conversaciones, evaluaciones, retroalimentaciones, progreso y reportes; mientras que Firebase Storage se utiliza para almacenar los archivos de audio generados durante las prácticas de speaking.

En cuanto al flujo de funcionamiento, el proceso inicia cuando el usuario interactúa con la aplicación móvil. La solicitud es enviada desde la capa de presentación hacia la API desarrollada en FastAPI, donde se ejecuta la lógica de negocio y el análisis mediante el modelo preentrenado de Machine Learning. Posteriormente, los resultados obtenidos son almacenados en Firebase y finalmente enviados nuevamente a la aplicación para que el usuario visualice la retroalimentación, el historial y el progreso de su aprendizaje.

6. Justificación de la arquitectura

La arquitectura seleccionada para el desarrollo de la aplicación corresponde a una **arquitectura en N capas**, debido a que permite organizar el sistema mediante la separación de responsabilidades entre la presentación, la lógica de negocio, el acceso a los datos y la persistencia de la información. Esta organización facilita el desarrollo, mantenimiento y escalabilidad del sistema, además de mejorar la reutilización del código y reducir el acoplamiento entre sus componentes.

- **La capa de presentación**, desarrollada en Flutter, se encarga exclusivamente de la interacción con el usuario, proporcionando interfaces intuitivas para el registro, acceso al sistema, interacción con el asistente virtual y consulta de resultados. Esta separación permite realizar modificaciones en la interfaz sin afectar la lógica de negocio.

- **La capa de negocio**, implementada mediante FastAPI, concentra las reglas de negocio y el procesamiento de las solicitudes provenientes de la aplicación móvil. En esta capa se integra el modelo preentrenado de Machine Learning, encargado de analizar la pronunciación y la gramática del estudiante para generar una retroalimentación automática, evitando que estas tareas sean ejecutadas directamente en el dispositivo móvil y mejorando así el rendimiento de la aplicación.
- **La capa de acceso a datos**, implementada mediante el patrón Repository, centraliza la comunicación entre la lógica de negocio y las fuentes de datos. Esta capa abstrae el acceso a Firebase y facilita futuras modificaciones en la infraestructura de almacenamiento sin afectar el funcionamiento del resto del sistema.
- **La capa de datos**, implementada con Firebase, permite gestionar la autenticación de usuarios mediante Firebase Authentication, almacenar la información en Cloud Firestore y conservar los archivos de audio en Firebase Storage, garantizando una administración segura y organizada de la información.

Adicionalmente, dentro de la capa de presentación se implementa el **patrón de diseño MVVM**, el cual permite separar la interfaz de usuario de la lógica de presentación, favoreciendo una mejor organización del código y facilitando su mantenimiento.

En conjunto, la arquitectura en N capas y los patrones de diseño utilizados permiten construir un sistema modular, mantenible, escalable y preparado para futuras ampliaciones, garantizando una adecuada integración entre la aplicación móvil, la lógica de negocio y la gestión de los datos.

7. Conclusión

El diseño arquitectónico implementado para la aplicación permitió establecer una estructura organizada y modular, basada en una arquitectura en N capas, que separa la presentación, la lógica de negocio, el acceso a los datos y la persistencia de la información. Esta organización facilita el mantenimiento, la escalabilidad y la evolución del sistema, permitiendo incorporar nuevas funcionalidades sin afectar significativamente su estructura.

La incorporación de los patrones de diseño MVVM y Repository complementa esta arquitectura al favorecer la separación de responsabilidades, mejorar la organización del

código y simplificar el acceso a las diferentes fuentes de datos. Asimismo, el uso de FastAPI para la lógica de negocio y Firebase para la autenticación y el almacenamiento de la información proporciona una integración eficiente entre los diferentes componentes del sistema.

En conjunto, la arquitectura propuesta responde a los requerimientos funcionales y no funcionales de la aplicación, permitiendo ofrecer una solución robusta, mantenible y preparada para futuras ampliaciones, capaz de apoyar el fortalecimiento de la destreza *speaking* mediante la integración de tecnologías de reconocimiento de voz y Machine Learning.